305141
Introduction to Internet of Things

Dr. Settha Tangkawanit

Department of Electrical and Computer Engineering

Faculty of Engineering

Naresuan University

Course Outline

Part 3: Back end and End user application

- Basic of Back end and End user application for IoT
- Web service concept
- Database
- Web application
- Mobile application

305141
Introduction to Internet of Things

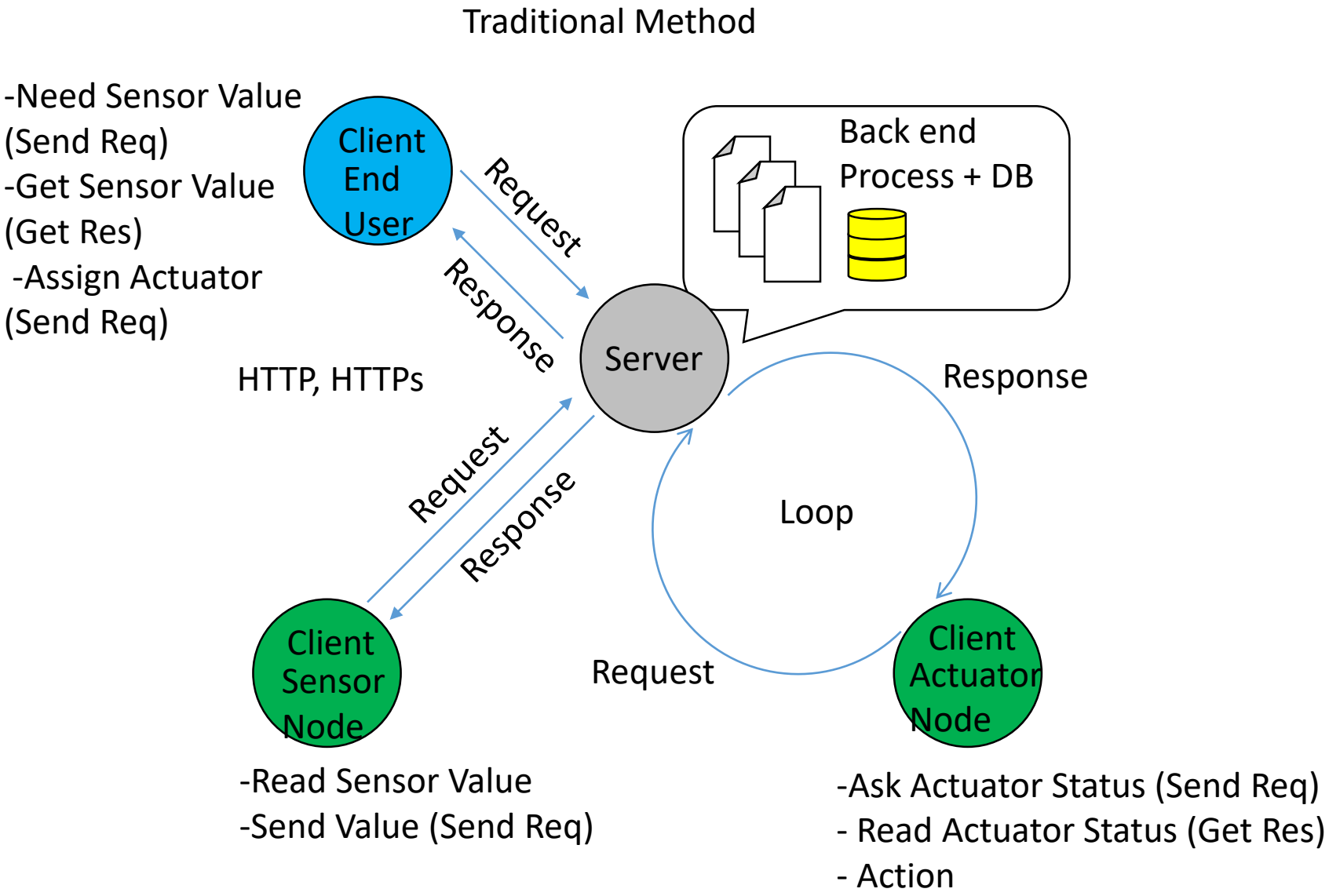
Chapter 05
Basic of Back-end and End user application

Dr. Settha Tangkawanit

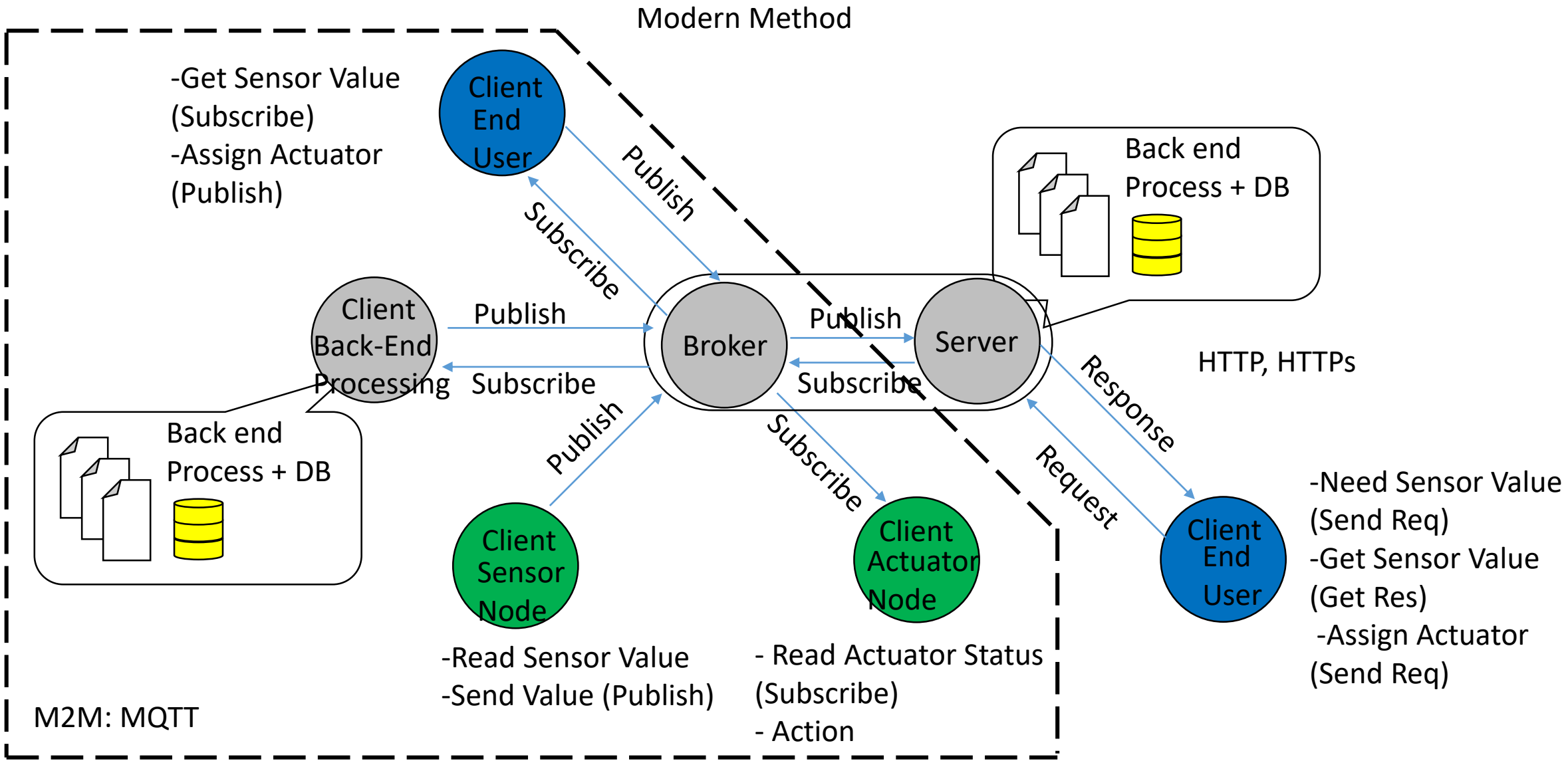
Department of Electrical and Computer Engineering

Faculty of Engineering, Naresuan University

IoT Basic Implementation



IoT Basic Implementation



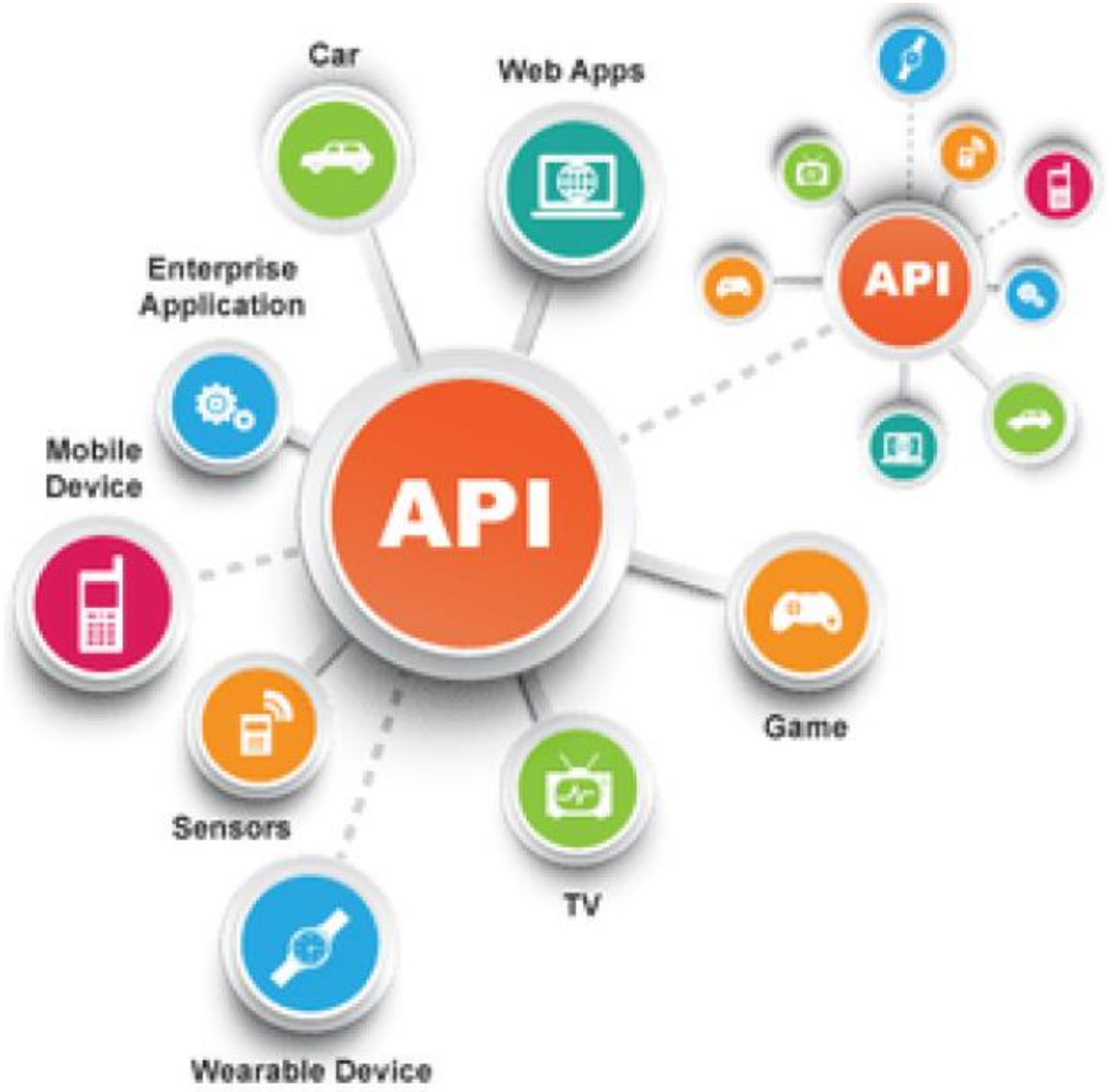
Back-end

- Back-end หรือระบบหลังบ้าน **มีหน้าที่ในการจัดการดูแลฐานข้อมูล การบริการ** และ**การจัดการเนื้อหา**ต่าง ๆ ที่แสดงผ่านตัวแอปพลิเคชั่นรวมถึงการสร้างการแจ้งเตือนต่าง ๆ มีความสำคัญ อย่างมากในการพัฒนาแอปพลิเคชั่นที่สามารถนำมาใช้งานได้จริงกับผู้ใช้งานจริง
- ซึ่งจะสามารถใช้งานแอปพลิเคชันได้**ผ่านเครือข่ายอินเตอร์เน็ต** และการแสดงผลข้อมูลผ่านแอปพลิเคชั่นก็ขึ้นอยู่กับส่วนหลังบ้านที่**มี การออกแบบฐานข้อมูล**และ**การส่งข้อมูล**ไปยังแอปพลิเคชั่นผ่าน RESTful Api

Web Service, API

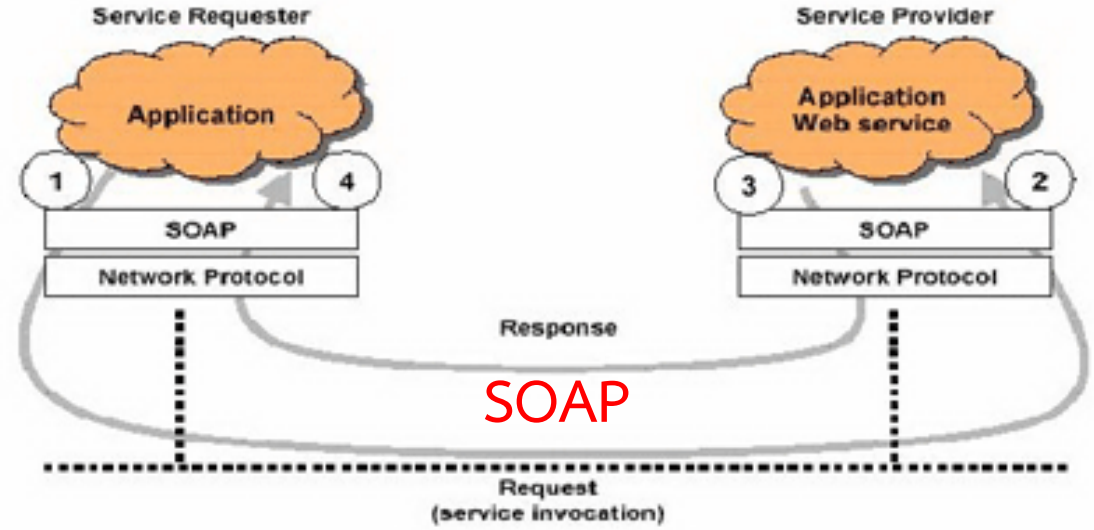
- **API หรือ Application Programming Interface** คือการเชื่อมต่อจากระบบหนึ่งไปสู่อีกระบบหนึ่ง **เพื่อให้ซอฟต์แวร์ภายนอกเข้าถึงและอัพเดทข้อมูลนั้นๆ ได้ แต่ยังอยู่ในขอบเขตที่ถูกกำหนดไว้** โดย **API เป็นตัวกลางที่จะทำให้คอยรับคำสั่งต่างๆ** ประมวลผลและกระทำข้อมูล**ส่งกลับคืนไปยังคนสั่งโดยอัตโนมัติ**
- เช่น แอปพลิเคชันต่างๆ ที่ใช้กันอยู่ในปัจจุบันนั่นเอง ทำให้บริษัทหรืออุตสาหกรรมต่างๆ เริ่มนำ API เข้ามาใช้งาน ทั้งนี้ก็เพื่อต้องการอำนวยความสะดวก ลดกำลังคน และลดความผิดพลาดลง
- ประโยชน์คือ **ช่วยเป็นสื่อกลางส่งข้อมูลข้าม Server** ลดการใช้กำลังคนด้วยการใช้เทคโนโลยี สื่อสารกับลูกค้าได้รวดเร็วขึ้น ไม่ต้องทำงานข้ามเว็บไซต์ **เพียงติดตั้ง API และใช้งานในเว็บไซต์ที่มี API เดียวก็พอ** ผู้ใช้มีความสะดวกสบายมากขึ้น ไม่ต้องเข้าโปรแกรมหรือ tools ในการทำงานให้ซับซ้อน

Web Service, API



Web Service, API

- Web APIs นิยมใช้กันมากในปัจจุบัน เพราะอยู่ในกลุ่มของ HTTP และขยายออกไปสู่รูปแบบ XML และ JSON ซึ่งโดนรวมแล้วก็คืออยู่บน web service เช่น
- SOAP (Simple Object Access Protocol) ใช้ XML format ส่งข้อมูล
- REST (Representational State Transfer) สามารถใช้ XML หรือ JSON format ส่งข้อมูล



Web Service, API

- SOAP (Simple Object Access Protocol) คือ โปรโตคอลมาตรฐานที่ใช้ใน Web Services เป็นโพรโทคอล (Protocol) ในการติดต่อแลกเปลี่ยนข้อมูลระหว่างกันของ web services เป็นโพรโทคอลการสื่อสาร ในระดับ Application Layer หรือในระดับแอปพลิเคชัน โดยอาศัยผ่านอินเทอร์เน็ตโพรโทคอล
- ซึ่งอาศัยรูปแบบของภาษา XML ทำให้ **Web services สามารถสื่อสารกันได้แม้ว่าจะอยู่บนเครื่องคอมพิวเตอร์คนละแพลตฟอร์ม** หรือพัฒนาด้วยภาษาโปรแกรมที่ต่างกันก็ตาม และนิยมใช้ HTTP เป็นโปรโตคอลร่วมสำหรับส่งผ่านข้อมูลบนระบบอินเตอร์เน็ต SOAP ทำงานร่วมกับโพรโทคอลได้หลายชนิด เช่น HTTP, SMTP, FTP, IIOP เป็นต้น

Web Service, API

- REST (Representational state transfer) คือ รูปแบบสถาปัตยกรรม ที่ใช้ประโยชน์จากเทคโนโลยี Web protocol เพื่อใช้ในการสร้าง Web Service โดยมีข้อกำหนดของ REST architectural style 7 ข้อ ประกอบไปด้วย
- Performance: ประสิทธิภาพในการโต้ตอบ
- Scalability: มีความสามารถในการปรับขยายได้ทำให้สามารถรองรับส่วนประกอบจำนวนมาก
- Simplicity: ความเรียบง่ายของอินเตอร์เฟสที่เหมือนกัน
- Modifiability: ความสามารถในการปรับเปลี่ยนส่วนประกอบเพื่อตอบสนองความต้องการที่เปลี่ยนแปลง (แม้ในขณะที่แอปพลิเคชันทำงานอยู่)
- Visibility: การมองเห็นการสื่อสารระหว่าง Component ผ่าน Service agents
- Portability: ง่ายต่อการโยกย้ายในส่วนของ Program และ Data
- Reliability: ความน่าเชื่อถือในการต้านทานความล้มเหลวในระบบ

Web Service, API

- RESTful Web Services (RWS) คือ Web Service ที่ใช้ REST architectural style เป็นที่รู้จักกันในชื่อ RESTful web services (RWS)
- โดยที่ RESTful Web service **อนุญาตให้ระบบ Request และเข้าถึง Resource บนเว็บ โดยใช้ชุดคำสั่งที่กำหนดเอาไว้ล่วงหน้า** โดยที่การโต้ตอบของระบบที่ใช้ REST จะอยู่บนพื้นฐานของ Hypertext Transfer Protocol (HTTP) Request จะส่งคำขอไปยัง URI ที่กำหนด และล้วงเอา response กลับมาเป็น Payload ในแบบ HTML, XML, JSON **หรือ format อื่น ๆ**
- **REST เป็นรูปแบบของสถาปัตยกรรมซอฟต์แวร์** ที่ใช้ประโยชน์จากเทคโนโลยี Web protocol เพื่อใช้ในการสร้าง Web Service **ขณะที่ RESTful มักใช้เพื่ออ้างถึง Web Service ที่ใช้สถาปัตยกรรม REST** ซึ่งสามารถกล่าวสรุปได้ว่า REST เป็นกระบวนทางสถาปัตยกรรม (architectural paradigm) ส่วน RESTful นั้นเป็นการใช้หรืออ้างถึงสถาปัตยกรรม (using that paradigm)

Web Service, API

Methods

GET

- ไคลเอ็นต์ใช้ GET เพื่อเข้าถึงทรัพยากรที่อยู่ที่ URL ที่ระบุบนเซิร์ฟเวอร์ ซึ่งสามารถแคชคำขอ GET และส่งพารามิเตอร์ในคำขอ RESTful API เพื่อสั่งให้เซิร์ฟเวอร์กรองข้อมูลก่อนส่ง

POST

- ไคลเอ็นต์ใช้ POST เพื่อส่งข้อมูลไปยังเซิร์ฟเวอร์ ซึ่งรวมถึงการแทนข้อมูลพร้อมกับคำขอ การส่งคำขอ POST เดียวกันหลายครั้งมีผลข้างเคียงเหมือนกับการสร้างทรัพยากรเดียวกันหลายครั้ง

PUT

- ไคลเอ็นต์ใช้ PUT เพื่ออัปเดตทรัพยากรที่มีอยู่บนเซิร์ฟเวอร์ การส่งคำขอ PUT เดียวกันหลายครั้งในบริการเว็บ RESTful จะให้ผลลัพธ์เหมือนกัน ซึ่งแตกต่างจาก POST

DELETE

- ไคลเอ็นต์ใช้คำขอ DELETE เพื่อลบทรัพยากรออก โดยคำขอ DELETE สามารถเปลี่ยนสถานะเซิร์ฟเวอร์ได้ อย่างไรก็ตาม หากผู้ใช้ไม่มีการรับรองความถูกต้องที่เหมาะสม คำขอก็จะล้มเหลว

Database

- **Database (ฐานข้อมูล)** คือ กลุ่มข้อมูลขนาดใหญ่ที่ถูกเก็บรวบรวมไว้ที่ใดที่หนึ่งโดยเป็น *ข้อมูลที่มีความสัมพันธ์กัน* ซึ่งถูก*จัดเก็บอย่างเป็นระบบ* โดยมีซอฟต์แวร์เข้ามาควบคุมกระบวนการใช้งาน การทำงาน หรือการประมวลผล ทำให้ผู้ใช้สามารถใช้ข้อมูลได้อย่างมีประสิทธิภาพ
- **ซอฟต์แวร์ที่ควบคุม** Database จะเรียกว่า **DBMS (Database Management System)** หรือ *ระบบจัดการฐานข้อมูล* มีหน้าที่*ช่วยให้ผู้ใช้เข้าถึงข้อมูลได้ง่ายสะดวก*และมีประสิทธิภาพ ทั้ง*การสร้างฐานข้อมูล* การ*แก้ไขฐานข้อมูล* หรือ*ค้นหาข้อมูล* ซึ่งช่วยลดการเก็บข้อมูลที่ซ้ำซ้อน และ รักษาความถูกต้องของข้อมูลภายใน Database
- ในปัจจุบันข้อมูลในยุคดิจิทัลกลายเป็นสินทรัพย์ สำคัญของธุรกิจในการบริหารจัดการและการใช้งาน เพื่อให้การทำงานขององค์กรมีประสิทธิภาพและ ศักยภาพสูงขึ้น แนวคิดการออกแบบ Database ให้ยืดหยุ่นและตอบโจทย์กับความต้องการขององค์ กรที่เปลี่ยนแปลงไปตามสถานการณ์อย่างรวดเร็วจึงมีความสำคัญ

Database: SQL

Structured Query Language

- คำสั่งที่ใช้บริหารจัดการฐานข้อมูลคือ **ภาษา programming** ที่ออกแบบมาเพื่อทำการจัดการข้อมูลที่อยู่ใน **relational database management system (RDBMS)** หรือก็คือไว้สำหรับ**ค้นหาข้อมูล เปลี่ยนแปลง เพิ่ม** และ **ลด** ข้อมูลที่ถูกเก็บอยู่ในฐานข้อมูลในรูปแบบตารางที่มีลักษณะเป็น **column** และ **row**
- เราเรียกข้อมูลเหล่านี้ว่าถูกเก็บอยู่ใน **table** ด้วยความสามารถของ SQL เรายังสามารถสร้างตารางขึ้นมาใหม่ (create) รวมถึง ลบ (drop) และเปลี่ยนแปลงค่า (alter) ของ table ได้

Database: SQL

- MySQL เป็น**ระบบจัดการฐานข้อมูลเชิงสัมพันธ์โอเพนซอร์ส**บนพื้นฐานของ SQL ซึ่ง MySQL ได้รับการ ออกแบบและปรับให้เหมาะสมสำหรับเว็บแอปพลิเคชันและสามารถ**ทำงานบนแพลตฟอร์มใดก็ได้ MySQL ทำงานเป็นดาต้าเบสเซิร์ฟเวอร์** และ**อนุญาตให้ผู้ ใช้หลายคนจัดการและสร้างฐานข้อมูล** จำนวนมาก มันเป็นองค์ประกอบสำคัญใน LAMP (Linux, Apache, MySQL และ PHP)
- MySQL กลายเป็นแพลตฟอร์มทางเลือกสำหรับนักพัฒนาเว็บไซต์และเว็บแอปพลิเคชัน เนื่องจากได้รับการ **ออกแบบมาเพื่อประมวลผลการค้นหาปริมาณมาก** โดยมีการใช้ **SQL ที่เป็นคำสั่งที่ใช้บริหารจัดการ database** ซึ่งเป็นภาษา programming ที่ออกแบบมาเพื่อทำการ จัดการข้อมูล **ค้นหาข้อมูล ปรับปรุง เปลี่ยนแปลง เพิ่ม และลบข้อมูลใน database**

Database: NoSQL

NoSQL

- NoSQL ย่อมาจาก Not Only SQL คือ เป็น Unstructured ของ Database แบบ SQL มันเป็นแนวทางการแก้ไขปัญหาของ Database ที่มีข้อมูลขนาดใหญ่และไม่มีรูปแบบที่ชัดเจน **โดยไม่จำเป็นต้องเก็บข้อมูลแบบ Table เดียวที่มีต้องข้อมูลเหมือนกันทั้งหมดในหนึ่ง Table** แต่สามารถจัดเก็บข้อมูลแบบได้หลายรูปแบบ

MongoDB เป็น open-source document database ประเภทหนึ่ง โดยเป็น database แบบ NoSQL Database จะไม่มีการใช้คำสั่ง SQL ไม่เน้นในการสร้างความสัมพันธ์ของข้อมูลแต่จะเป็นรูปแบบโครงสร้างที่เจ้าของ NoSQL สร้างขึ้นมาเองและจัดเก็บข้อมูลเป็นแบบ JSON (JavaScript Object Notation) ซึ่งจะเก็บค่าเป็น key และ value โดยจุดเด่นอยู่ที่ความเร็วในการทำงานเป็นหลัก คิวรี่ข้อมูลได้เร็วขึ้น การทำงานในส่วนของ database จะลดลง แต่จะไปเน้นการทำงานในส่วนของโปรแกรมที่พัฒนาขึ้นมาแทน โดย database ประเภทนี้ จะเหมาะกับข้อมูลขนาดใหญ่ ที่ไม่ซับซ้อน การทำงานที่ไม่หนักมาก สามารถทำงานกับระบบที่เป็นการทำงานแบบเรียลไทม์ (Real Time) ได้ดี

Database: NoSQL

MongoDB

- เป็น open-source document database ประเภทหนึ่ง โดยเป็น database แบบ NoSQL Database จะไม่มีการใช้คำสั่ง SQL ไม่เน้นในการสร้างความสัมพันธ์ของข้อมูล แต่จะเป็นรูปแบบโครงสร้างที่เจ้าของ NoSQL สร้างขึ้นมาเองและจัดเก็บข้อมูลเป็นแบบ JSON (JavaScript Object Notation)
- ซึ่งจะเก็บค่าเป็น key และ value โดยจุดเด่นอยู่ที่ความเร็วในการทำงานเป็นหลัก คิวรี่ข้อมูลได้เร็วขึ้น การทำงานในส่วนของ database จะลดลง แต่จะไปเน้นการทำงานในส่วนของโปรแกรมที่พัฒนาขึ้นมาแทน โดย database ประเภทนี้ จะเหมาะกับข้อมูลขนาดใหญ่ ที่ไม่ซับซ้อน การทำงานที่ไม่หนักมาก สามารถทำงานกับระบบที่เป็นการทำงานแบบเรียลไทม์ (Real Time) ได้ดี